MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 10 — Les Dictionnaires

La structure clé-valeur — chercher vite, trouver juste

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

8 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 10 — Les Dictionnaires **2**
0.1 Créer un dictionnaire 2
0.2 Accéder aux valeurs 3
0.3 Modifier un dictionnaire 3
0.4 Parcourir un dictionnaire 4
0.5 Les dictionnaires imbriqués 5
0.6 La dict comprehension 6
0.7 Méthodes utiles du dictionnaire 6
0.8 Cas pratiques — Comptage et groupement 7
0.9 Ce qui m'a le plus surpris dans ce chapitre 8
0.10 Résumé du Chapitre 10 9

Chapitre 10 — Les Dictionnaires

À retenir

Un dictionnaire est une collection de paires **clé : valeur**. Contrairement aux listes indexées par des entiers, les dictionnaires permettent d'accéder aux valeurs via des **clés significatives**. C'est la structure de données la plus proche du monde réel — un carnet d'adresses, une fiche client, un objet JSON. Rapide, flexible et omniprésent en Python.

Créer un dictionnaire

Un dictionnaire est délimité par des **accolades** `{}` et chaque entrée suit le format `clé: valeur`.

Créer des dictionnaires

```python
# Dictionnaire simple
personne = {
    "prenom": "Alice",
    "nom":    "Dupont",
    "age":    25,
    "ville":  "Paris"
}

# Dictionnaire de types varies
profil = {
    "pseudo":    "alice42",
    "score":     1500,
    "actif":     True,
    "langages":  ["Python", "JavaScript"],
    "niveau":    4.5
}

# Dictionnaire vide
config = {}
config2 = dict()

# Avec dict() et des arguments nommes
point = dict(x=10, y=20, z=0)
print(point)   # {'x': 10, 'y': 20, 'z': 0}

# Depuis une liste de tuples
paires = [("rouge", "#FF0000"), ("vert", "#00FF00")]
couleurs = dict(paires)
print(couleurs)   # {'rouge': '#FF0000', 'vert': '#00FF00'}
```

```python

print(len(personne))    # 4
```

Accéder aux valeurs

Accès par clé et get()

```python
personne = {"prenom": "Alice", "age": 25, "ville": "Paris"}

# Acces direct par cle      [ ]
print(personne["prenom"])    # Alice
print(personne["age"])       # 25

#      Cl   inexistante      KeyError
# print(personne["email"])

#      get()      acces securisr avec valeur par defaut
print(personne.get("ville"))             # Paris
print(personne.get("email"))             # None
print(personne.get("email", "N/A"))      # N/A
print(personne.get("age", 0))            # 25

# Verifier si une cle existe
print("prenom" in personne)     # True
print("email" in personne)      # False
print("age" not in personne)    # False
```

Erreur fréquente

Accéder à une clé inexistante avec `dict["clé"]` lève une `KeyError` et plante le programme. Toujours préférer `dict.get("clé", valeur_défaut)` quand on n'est pas certain que la clé existe. C'est l'une des erreurs les plus fréquentes avec les dictionnaires.

Modifier un dictionnaire

Ajouter, modifier, supprimer

```python
personne = {"prenom": "Alice", "age": 25}

# Modifier une valeur existante
personne["age"] = 26
print(personne)    # {'prenom': 'Alice', 'age': 26}

# Ajouter une nouvelle paire cle-valeur
personne["ville"] = "Lyon"
```

```python
personne["email"] = "alice@exemple.com"
print(personne)
# {'prenom': 'Alice', 'age': 26, 'ville': 'Lyon', 'email': ...}

# update()      fusionner avec un autre dict
mises_a_jour = {"age": 27, "pays": "France"}
personne.update(mises_a_jour)
print(personne["age"])     # 27
print(personne["pays"])    # France

# Supprimer avec del
del personne["email"]

# pop()      supprimer et recuperer la valeur
age = personne.pop("age")
print(age)         # 27
print(personne)    # sans "age"

# pop() avec valeur par defaut si cle absente
val = personne.pop("inexistant", "absent")
print(val)    # absent

# clear()      vider le dictionnaire
personne.clear()
print(personne)    # {}
```

Parcourir un dictionnaire

keys(), values(), items()

```python
etudiant = {
    "prenom": "Bob",
    "note_maths": 16,
    "note_info": 18,
    "note_anglais": 14
}

# Parcourir les CLES (comportement par defaut)
for cle in etudiant:
    print(cle)
# prenom, note_maths, note_info, note_anglais

# keys()      explicite et lisible
for cle in etudiant.keys():
    print(cle)

# values()      les valeurs uniquement
for valeur in etudiant.values():
```

```
    print(valeur)
# Bob, 16, 18, 14

# items()      cles ET valeurs ensemble      (le plus utilise)
for cle, valeur in etudiant.items():
    print(f"{cle:20} : {valeur}")

# Calculer la moyenne des notes
notes = [v for k, v in etudiant.items() if k.startswith("note")
    ]
moyenne = sum(notes) / len(notes)
print(f"Moyenne : {moyenne:.2f}")    # 16.00
```

À retenir

`.items()` est la méthode la plus utilisée pour parcourir un dictionnaire. Elle retourne des paires `(clé, valeur)` que l'on peut déballer directement dans la boucle `for`. C'est la façon **pythonique** d'itérer sur un dictionnaire.

Les dictionnaires imbriqués

Un dictionnaire peut contenir d'autres dictionnaires — permettant de modéliser des **structures complexes**.

Dictionnaires imbriqués

```
# Base de donnees d'utilisateurs
utilisateurs = {
    "alice": {
        "nom":    "Alice Dupont",
        "age":    25,
        "email":  "alice@exemple.com",
        "scores": [85, 92, 78]
    },
    "bob": {
        "nom":    "Bob Martin",
        "age":    30,
        "email":  "bob@exemple.com",
        "scores": [90, 88, 95]
    }
}

# Acces imbrique
print(utilisateurs["alice"]["nom"])       # Alice Dupont
print(utilisateurs["bob"]["age"])         # 30
print(utilisateurs["alice"]["scores"])    # [85, 92, 78]
print(utilisateurs["alice"]["scores"][1])  # 92
```

```python
# Modifier une valeur imbriquee
utilisateurs["alice"]["age"] = 26
utilisateurs["bob"]["scores"].append(100)

# Parcourir tous les utilisateurs
for pseudo, info in utilisateurs.items():
    moy = sum(info["scores"]) / len(info["scores"])
    print(f"{pseudo:10}     {info['nom']:15}     Moy: {moy:.1f}
        ")
```

La dict comprehension

Comme les listes, les dictionnaires ont leur propre syntaxe de comprehension.

Dict comprehension

```python
# Syntaxe : {cle: valeur for element in iterable if condition}

# Carres des nombres de 1 a 5
carres = {n: n**2 for n in range(1, 6)}
print(carres)
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Inverser cles et valeurs
original = {"a": 1, "b": 2, "c": 3}
inverse = {v: k for k, v in original.items()}
print(inverse)   # {1: 'a', 2: 'b', 3: 'c'}

# Filtrer les notes >= 10
notes = {"Alice": 15, "Bob": 8, "Charlie": 12, "Diana": 5}
admis = {nom: note for nom, note in notes.items() if note >=
    10}
print(admis)   # {'Alice': 15, 'Charlie': 12}

# Normaliser des cles (tout en minuscules)
donnees = {"Prenom": "Alice", "AGE": 25, "VILLE": "Paris"}
propre = {k.lower(): v for k, v in donnees.items()}
print(propre)   # {'prenom': 'Alice', 'age': 25, 'ville': '
    Paris'}
```

Méthodes utiles du dictionnaire

setdefault(), fromkeys(), copy()

```python
config = {"debug": False, "version": "1.0"}

# setdefault()     ajoute la cle SEULEMENT si elle n'existe pas
```

```python
config.setdefault("langue", "fr")
print(config["langue"])     # fr

config.setdefault("debug", True)    # "debug" existe deja
print(config["debug"])              # False  (inchange !)

# fromkeys()      creer un dict depuis une liste de cles
cles = ["nom", "age", "email"]
fiche = dict.fromkeys(cles, "")
print(fiche)    # {'nom': '', 'age': '', 'email': ''}

fiche_defaut = dict.fromkeys(cles, None)
print(fiche_defaut)

# copy()      copie superficielle
original = {"a": 1, "b": 2}
copie = original.copy()
copie["c"] = 3
print(original)    # {'a': 1, 'b': 2}       inchange
print(copie)       # {'a': 1, 'b': 2, 'c': 3}

# Fusionner deux dicts      operateur | (Python 3.9+)
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 99, "c": 3}
fusion = dict1 | dict2
print(fusion)    # {'a': 1, 'b': 99, 'c': 3}
```

Note importante

L'opérateur `|` pour fusionner des dictionnaires est disponible depuis **Python 3.9**. En cas de clé dupliquée, le dictionnaire de **droite** l'emporte. C'est une syntaxe élégante et moderne qui remplace le `dict.update()` dans bien des cas.

Cas pratiques — Comptage et groupement

Compter les occurrences

```python
# Compter les lettres d'un mot
mot = "mississippi"
compteur = {}

for lettre in mot:
    compteur[lettre] = compteur.get(lettre, 0) + 1

print(compteur)
# {'m': 1, 'i': 4, 's': 4, 'p': 2}

# Trier par frequence decroissante
```

```python
trie = dict(sorted(compteur.items(),
                   key=lambda x: x[1],
                   reverse=True))
print(trie)
# {'i': 4, 's': 4, 'p': 2, 'm': 1}

# Grouper des etudiants par mention
etudiants = [
    ("Alice", 16), ("Bob", 9), ("Charlie", 13),
    ("Diana", 18), ("Eve", 11), ("Frank", 7)
]

groupes = {}
for nom, note in etudiants:
    mention = "Admis" if note >= 10 else "Recale"
    groupes.setdefault(mention, []).append(nom)

print(groupes)
# {'Admis': ['Alice', 'Charlie', 'Diana', 'Eve'],
#  'Recale': ['Bob', 'Frank']}
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `get()` vs `[]` — une différence qui peut tout changer. J'ai longtemps utilisé `dict["clé"]` partout, sans réaliser que chaque accès à une clé inexistante est un programme qui plante. `get()` est la version défensive et professionnelle. Depuis que je l'ai adopté, mes programmes sont infiniment plus robustes. La règle que je retiens : **si on n'est pas sûr que la clé existe, on utilise `get()`**.

2. Le pattern `dict.get(clé, 0) + 1` pour compter. Cette ligne `compteur[lettre] = compteur.get(lettre, 0) + 1` m'a d'abord semblé dense. Mais une fois décomposée, c'est d'une élégance redoutable : si la clé existe, on incrémente. Sinon, on part de 0. Pas de `if`, pas de vérification explicite. C'est le pattern de comptage pythonique par excellence.

3. Les dictionnaires sont ordonnés depuis Python 3.7. Je pensais que les dictionnaires étaient non-ordonnés — c'était vrai avant Python 3.7. Depuis, l'ordre d'insertion est **garanti**. Cette évolution du langage m'a rappelé que Python évolue, et qu'il est important de toujours vérifier la version ciblée quand on lit de la documentation.

Résumé du Chapitre 10

Ce que j'ai appris dans ce chapitre :

- Un dictionnaire stocke des paires **clé : valeur** entre accolades `{}`.
- Accès sécurisé avec `.get(clé, défaut)` — jamais de `KeyError`.
- Modifier : `dict[clé] = val`, `update()`, `pop()`, `del`.
- Parcourir avec `.keys()`, `.values()`, `.items()`.
- **Dict comprehension** : `{k: v for k, v in items if condition}`.
- `setdefault()` — ajouter une clé seulement si elle est absente.
- Fusionner avec `|` (Python 3.9+) ou `update()`.
- Les dicts sont **ordonnés** depuis Python 3.7 — l'ordre d'insertion est conservé.

"Un dictionnaire, c'est le monde réel modélisé en Python. Chaque objet autour de nous peut s'écrire clé : valeur."